UNIVERSITI TEKNOLOGI PETRONAS

DEPARTMENT OF COMPUTING

TEB2053: EMBEDDED SYSTEM

JANUARY SEMESTER 2026

PROJECT REPORT

Project Title: QuakeGuard Intelligent Earthquake Early Warning System

Name	Student ID	Course
Low Ren Jing	22011246	Bachelor of Computer Science
Wilfred Wong Ying Xuan	22011422	Bachelor of Computer Science
Mahesh A/L Vigneswaran	22010786	Bachelor of Computer Science
Loo Hong Sheng	22011393	Bachelor of Computer Science
Haw Jean Yung	22011237	Bachelor of Computer Science

Contents

1.0 Project Introduction

Earthquakes are one of the most devastating natural disasters, capable of causing severe damage to infrastructure and endangering human lives. In many cases, the damage caused by earthquakes is not limited to structural collapse but also includes secondary hazards such as gas leaks, fires, and difficulties in evacuating buildings. These risks highlight the importance of having reliable systems that can detect seismic activity and provide early warnings to occupants of buildings.

Advancements in embedded systems technology have enabled the development of compact and efficient monitoring systems that can operate in real time. By integrating sensors, microcontrollers, and automated response mechanisms, embedded systems can be used to monitor environmental conditions and trigger safety actions when dangerous events occur. Such systems are particularly valuable in disaster management applications, where rapid detection and response are critical to minimizing risks.

This project presents QuakeGuard, an Earthquake Early Warning System designed to improve building safety during seismic events. The system is developed using the Arduino Uno R3 microcontroller platform and simulated using Tinkercad Circuits. The system continuously monitors vibration signals and detects potential earthquake activity using an interrupt-driven detection mechanism. Once a vibration event is detected, the system analyzes the signals within a defined time window to determine the severity of the earthquake.

Depending on the detected severity level, the system provides real-time alerts using multiple output devices such as LEDs, a buzzer, and a 16×2 LCD display. In addition, the system can execute automated safety responses, including shutting off the gas valve and unlocking emergency doors to support safe evacuation procedures. These features allow the system to provide both early warnings and protective actions to reduce potential risks during an earthquake.

Overall, the QuakeGuard system demonstrates how embedded systems can be applied in disaster preparedness and building safety. By combining real-time detection, intelligent event classification, and automated safety mechanisms, the system aims to provide an efficient and responsive earthquake early warning solution.

2.0 Problem Statement

2.1 Global Context

Earthquakes cause over 50,000 deaths annually worldwide, with significant economic losses and infrastructure damage. The immediate dangers of structural collapse are compounded by secondary hazards including:

- Post-earthquake fires from ruptured gas lines (30% earthquake casualties)
- Building entrapment due to jammed electronic locks (15% of victims)
- Delayed evacuation from lack of early warning systems

Studies show that early warning systems can reduce earthquake fatalities by up to 60%, as demonstrated by Japan's nationwide seismic network which provides 10-30 seconds of warning before major shaking arrives.

2.2 Technical Challenges

Existing earthquake detection systems face several limitations:

- Polling-based sensors may miss rapid vibration events due to sampling delays
- Binary detection (earthquake vs. no earthquake) provides insufficient context for proportional response
- Fixed sensitivity thresholds don't account for different building structural characteristics
- Blocking code architectures freeze system responsiveness during critical actuator movements
- Manual reset procedures prevent automated recovery after false alarms

2.3 Proposed Solution

This project addresses these challenges through a comprehensive embedded system that combines interrupt-driven detection (zero-latency response), intelligent pulse-counting analysis (distinguishes real earthquakes from false triggers), dynamic sensitivity adjustment (adaptable to building types), and non-blocking state machine control (maintains responsiveness during emergency protocols). The system provides graduated responses from minor tremor logging to full emergency activation including gas shutoff and door unlocking, with real-time visual, audio, and text feedback to occupants.

3.0 Scope

3.1 Inclusions

This project implements:

- Hardware interrupt-based earthquake detection with 50ms debouncing
- Pulse-counting analysis over a 5-second window for severity classification
- Dynamic threshold adjustment via potentiometer (inverse sensitivity mapping)
- 9-state finite state machine with non-blocking state transitions
- Graduated response system: Minor (log only), Moderate (warning), Major (emergency protocols)
- Automated gas valve shutoff using servo motor control
- Automatic emergency door unlocking system
- Multi-modal feedback: Color-coded LEDs, frequency-modulated buzzer, 16x2 LCD display
- 10-second countdown warning for moderate and major earthquakes
- Manual reset capability with fail-safe state management
- System self-test mode for validation and demonstration
- Serial monitor debugging with state transitions and event logging

3.2 Exclusions

This project does not include:

- Real seismographic sensors (simulated with push button in Tinkercad)
- Wireless communication or network connectivity
- Timestamp logging or real-time clock integration
- Persistent data storage (EEPROM/SD card)
- Multi-building coordination or centralized control
- Battery backup or uninterruptible power supply

3.3 Limitations

- System operates within Tinkercad simulation environment constraints
- Vibration detection simulated through manual button presses rather than actual seismic sensors
- Servo power draw limited to Arduino 5V output (real implementation would require external power)
- Event counter resets on power cycle (no persistent memory)

4.0 Goals

4.1 Primary Goals

1. Demonstrate Finite State Machine Design:

- Implement a robust 9-state FSM that manages system behavior from standby through emergency response and reset, with clear state transitions and documented logic flow.

2. Implement Real-Time Event Detection:

- Utilizing hardware interrupts to achieve zero-latency earthquake detection, ensuring no vibration events are missed during system operation.

3. Create Intelligent Classification Algorithm:

- Develop a pulse-counting system that analyzes vibration frequency over a 5-second window to distinguish minor tremors from dangerous earthquakes.

4. Execute Automated Safety Protocols:

- Implement non-blocking servo control for gas valve shutoff and emergency door unlocking during major earthquake events.

5. Provide Multi-Modal User Feedback:

- Integrate visual (LEDs), audio (buzzer), and textual (LCD) feedback to communicate system status and urgency levels to occupants.

4.2 Secondary Goals

- Demonstrate adaptive system behavior through user-configurable sensitivity adjustment
- Implement fail-safe manual reset procedures to prevent auto-recovery during aftershocks
- Create comprehensive serial debugging output for system monitoring and troubleshooting
- Develop self-test capability for system validation and demonstration purposes

5.0 Functional and Non-Functional Requirements

5.1 Functional Requirements

5.1.1: Interrupt-Driven Detection

- System shall use hardware interrupt on Pin 2 (RISING edge) for earthquake detection
- Interrupt Service Routine (ISR) shall include 50ms debounce protection
- ISR shall set volatile Boolean flag atomically
- Detection latency shall not exceed 100 microseconds

5.1.2: Pulse Counting Analysis

- System shall count vibration pulses over a fixed 5-second analysis window
- Each button presses during analysis increments pulse counter
- Pulse counter shall reset to 0 after classification
- LCD shall display remaining analysis time: 'Time left: Xs'

5.1.3: Dynamic Threshold Classification

- System shall read sensitivity potentiometer (Pin A0, 10-bit ADC: 0-1023)
- Moderate earthquake threshold: map(sensitivityValue, 0, 1023, 15, 4)
- Major earthquake threshold: map(sensitivityValue, 0, 1023, 25, 8)
- Classification: pulses < moderate → MINOR; moderate ≤ pulses < major → MODERATE; pulses ≥ major → MAJOR

5.1.4: Earthquake Response Actions

- Minor: Yellow LED ON, single 200ms beep (1000 Hz), display for 3 seconds, return to STANDBY
- Moderate: Orange LED ON, 3-beep pattern (1500 Hz), 10-second countdown, return to STANDBY
- Major: Red LED ON, continuous siren (2000 Hz), 10-second countdown, trigger EMERGENCY_ACTIVE

5.1.5: Emergency Safety Protocols

- Gas valve servo (Pin 9): Close to 90° at t=0ms upon entering EMERGENCY_ACTIVE
- Door lock servo (Pin 10): Unlock to 180° at t=300ms after gas valve
- Non-blocking sequence using emergencyStep variable (0, 1, 2)
- LCD displays "Gas:OFF Door:OPEN" / "EMERGENCY MODE"
- Transition to SAFE_MODE after 2 seconds

5.1.6: System Reset

- SAFE_MODE: Red LED ON, very slow beep (300ms ON/1700ms OFF), await Reset button
- SYSTEM_RESETTING: Non-blocking sequence using resetStep variable
- t=0ms: Turn off LEDs, open gas valve (0°)
- t=300ms: Lock door (0°)
- t=1500ms: Clear variables, return to STANDBY

5.1.7: User Interface

- LCD 16x2 parallel 4-bit mode: Pins 12 (RS), 11 (Enable), 13 (D4), A1 (D5), A2 (D6), A3 (D7)
- STANDBY display: "EQ Monitor: SAFE" / "Sensitivity: XX%"
- ANALYZING display: "Analyzing..." / "Time left: Xs"
- Update interval: Maximum 500ms to prevent flicker

5.1.8: Manual Controls

- Reset Button (Pin 3): Software polling, 50ms debounce, triggers SYSTEM_RESETTING from SAFE_MODE
- Test Button (Pin 4): Software polling, 50ms debounce, forces tremorPulseCount = 30 (guarantees MAJOR)
- Sensitivity Pot (Pin A0): Inverse mapping, higher value = more sensitive, display as 0-100%

5.1.9: Event Logging

- Tremor counter increments on each detection, persists across states
- Serial Monitor outputs: state transitions, pulse counts, thresholds, classifications, servo actions
- Baud rate: 9600

5.2 Non-Functional Requirements

5.2.1 Response Time and Performance

Metric	Requirement	Justification
Interrupt Latency	< 100 microseconds	Hardware interrupt ensures immediate detection
State Transition Time	< 10 milliseconds	FSM processes in single loop iteration
LCD Update Interval	500ms minimum	Prevents flicker and bus congestion
Servo Settling Time	300ms per movement	Mechanical components reach position
Emergency Protocol Total	1.2 seconds	Non-blocking ensures responsiveness

5.2.2: Fail-Safe Behaviors

- Power Loss: Servos default to safe positions (gas open, doors locked) on restart
- False Triggers: 5-second analysis window filters single bumps
- Aftershocks: System can detect new earthquakes during EMERGENCY_ACTIVE state
- Manual Override: Reset button accessible in critical states

5.2.3: Debouncing Protection

- Hardware Debounce: 50ms minimum between ISR triggers
- Software Debounce: 50ms edge detection delay for buttons
- Purpose: Prevents mechanical contact bounce from causing false counts

5.2.4: No Blocking Operations

- No delay() calls in state handlers (exception: 2-second delay in setup() only)
- All timing uses millis() comparison for non-blocking behavior
- Servo sequences use state-based progression (emergencyStep, resetStep)

5.2.5: Usability

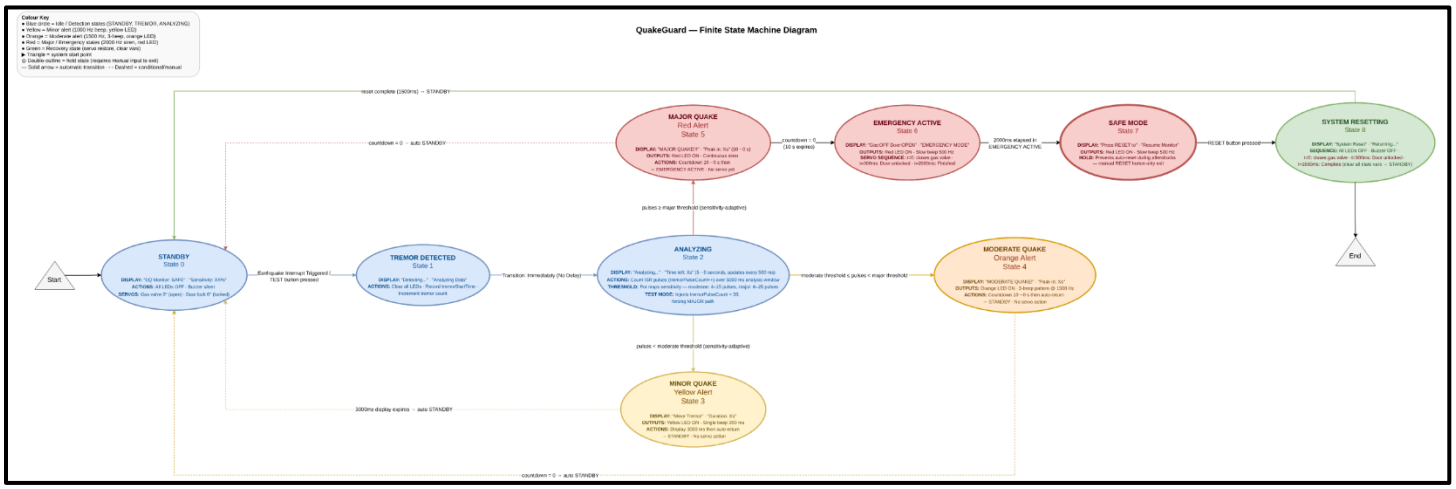
- LCD messages use action-oriented language ("Press RESET to Resume Monitor")
- Color-coded LEDs: Yellow (informational), Orange (warning), Red (critical)
- Sensitivity displayed as percentage (0-100%) in STANDBY mode
- Clockwise pot rotation increases sensitivity (intuitive direction)

5.2.6: Tinkercad Compatibility

- All components available in Tinkercad standard library
- Uses only Arduino core libraries (LiquidCrystal.h, Servo.h)
- Vibration sensor simulated with push button
- Arduino 5V sufficient for 2 servos in simulation

6.0 Finite State Machine (FSM) Diagram

6.1 Diagram



Link to diagram:
https://drive.google.com/file/d/1Wt4H_rsCSLtYlBeOjciEhysIVCgibfjW/view?usp=sharing

6.2 State Descriptions

State 0: STANDBY

- Normal monitoring mode. System displays current sensitivity setting and waits for earthquake detection.
- **Lcd:** "EQ Monitor: SAFE" / "Sensitivity: XX%"
- **Leds:** All OFF
- **Buzzer:** Silent
- **Exit:** Earthquake interrupt OR Test button pressed
- **Next:** TREMOR_DETECTED

State 1: TREMOR_DETECTED

- Earthquake interrupt has triggered. Brief transitional state before analysis begins.
- **Lcd:** "Detecting..." / "Analyzing Data"
- **Actions:** Clear all LEDs, start timer, increment tremor count
- **Exit:** Immediate automatic transition
- **Next:** ANALYZING

State 2: ANALYZING

- 5-second pulse counting window. Each earthquake button press during this window increments the pulse counter.
- **Lcd:** "Analyzing..." / "Time left: Xs" (countdown from 5 to 0)
- **Process:** Count vibration pulses, calculate dynamic thresholds based on sensitivity
- **Exit:** tremorDuration ≥ 5000ms (5 seconds elapsed)
- **Next:** MINOR_QUAKE / MODERATE_QUAKE / MAJOR_QUAKE (based on pulse count)

State 3: MINOR_QUAKE

- Low severity earthquake detected. Informational alert only, no safety actions.
- **Led:** Yellow LED ON
- **Buzzer:** Single 200ms beep (1000 Hz)
- **Lcd:** "Minor Tremor" / "Duration: Xs"
- **Exit:** 3 seconds elapsed
- **Next:** STANDBY (automatic return)

State 4: MODERATE_QUAKE

- Medium severity earthquake. Warning alerts with countdown timer. No safety actions.
- **Led:** Orange LED ON
- **Buzzer:** 3-beep pattern (1500 Hz), repeating every 2 seconds
- **Lcd:** "MODERATE QUAKE!" / "Peak in: Xs" (10-second countdown)
- **Exit:** countdownSeconds ≤ 0
- **Next:** STANDBY (automatic return)

State 5: MAJOR_QUAKE

- High severity earthquake. Critical alerts with countdown before emergency protocol activation.
- **Led:** Red LED ON
- **Buzzer:** Continuous siren (2000 Hz)
- **Lcd:** "MAJOR QUAKE!!!" / "Peak in: Xs" (10-second countdown)
- **Exit:** countdownSeconds ≤ 0
- **Next:** EMERGENCY_ACTIVE (trigger safety protocols)

State 6: EMERGENCY_ACTIVE

- Executing emergency safety protocols via non-blocking servo control.
- **Led:** Red LED remains ON
- **Buzzer:** Slow beep pattern (500 Hz, 500ms ON/OFF)
- **Lcd:** "Gas:OFF Door:OPEN" / "EMERGENCY MODE"
- **Actions:** t=0ms: Close gas valve; t=300ms: Unlock doors
- **Exit:** 2 seconds elapsed
- **Next:** SAFE_MODE

State 7: SAFE_MODE

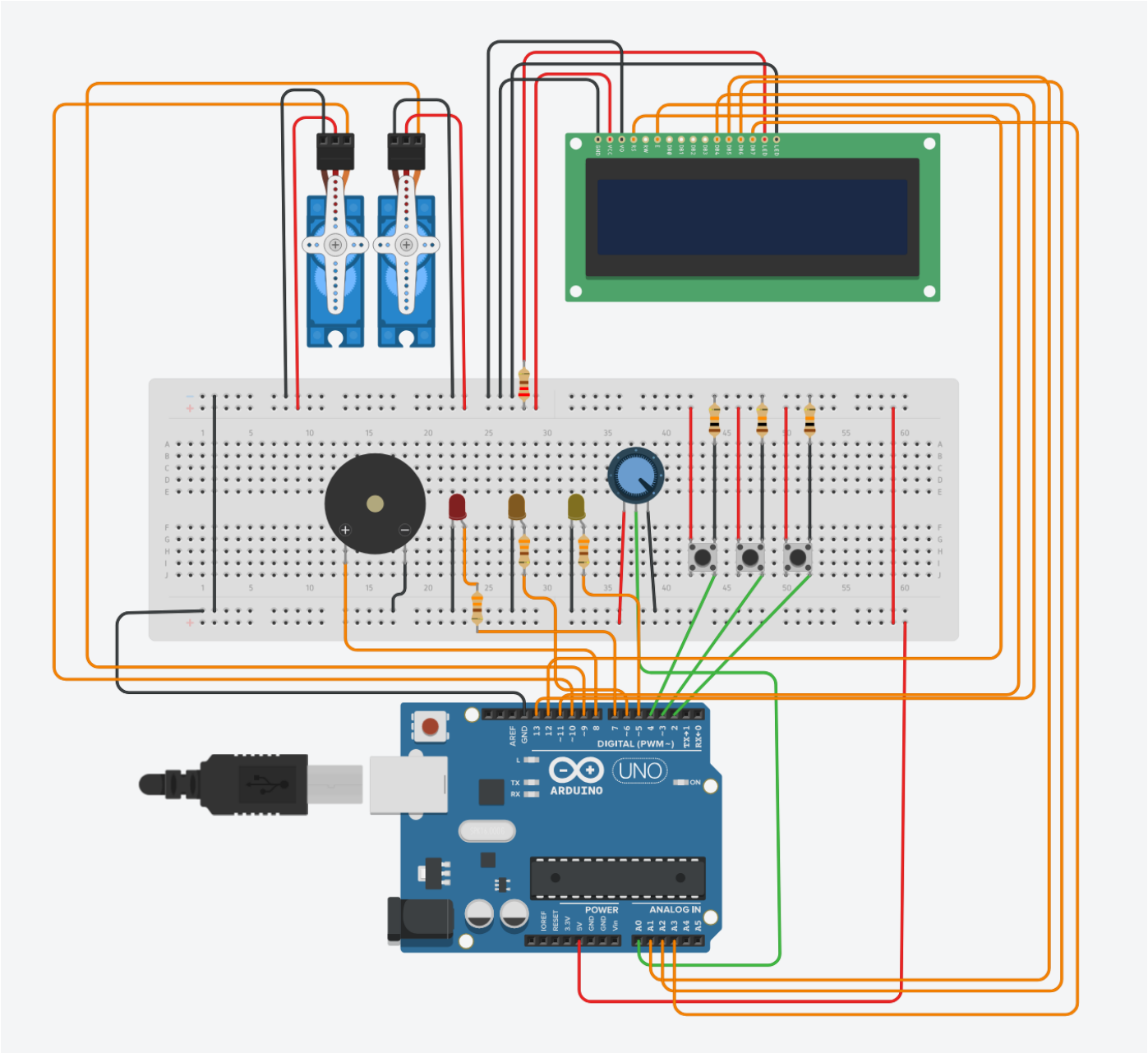
- Emergency protocols active. Awaiting manual inspection and reset.
- **Led:** Red LED remains ON
- **Buzzer:** Very slow beep (500 Hz, 300ms ON / 1700ms OFF)
- **Lcd:** "Press RESET to" / "Resume Monitor"
- **Exit:** RESET button pressed
- **Next:** SYSTEM_RESETTING

State 8: SYSTEM_RESETTING

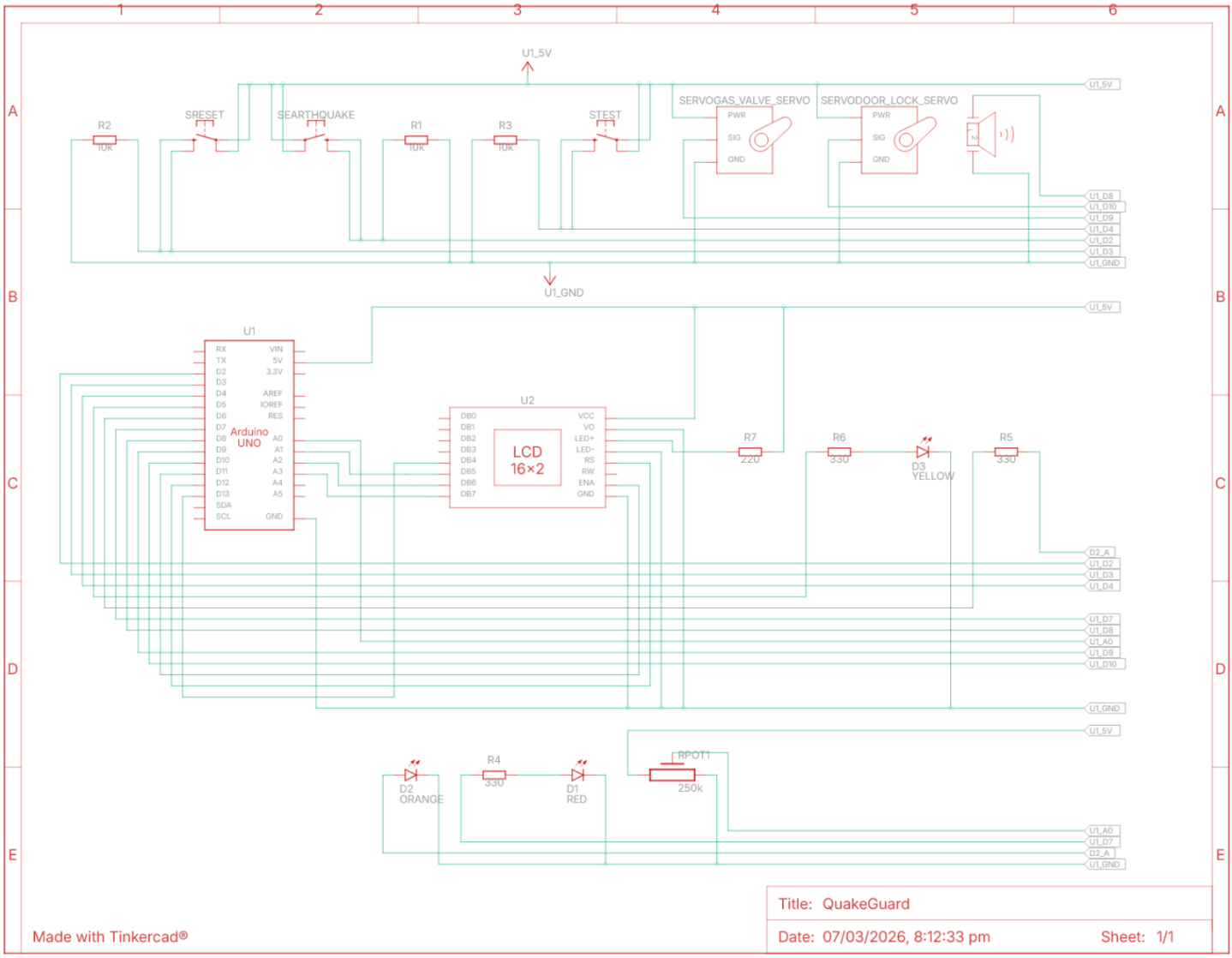
- Non-blocking reset sequence. Returns servos to safe positions and clears flags.
- **Led:** All LEDs OFF
- **Buzzer:** Silent
- **Lcd:** "System Reset" / "Returning..."
- **Actions:** t=0ms: Open gas valve; t=300ms: Lock doors; t=1500ms: Reset complete
- **Exit:** 1.5 seconds elapsed
- **Next:** STANDBY (normal operation restored)

7.0 Tinkercad Circuit View and Schematic View

7.1 Circuit View



7.2 Schematic View



8.0 Component List

Component	Quantity	Model/Type	Purpose
Arduino Uno R3	1	ATmega328P	Main controller
LCD 16x2	1	Parallel 4-bit mode	Status display
Positional Micro Servo	2	0-180° rotation	Gas valve, door lock
Push Button	3	Normally-open momentary	EQ, Reset, Test inputs
Potentiometer	1	10kΩ linear taper	Sensitivity adjustment
LED (5mm)	3	Yellow, Orange, Red	Severity indicators
Piezo Buzzer	1	Active buzzer	Audio alerts
Resistor 220Ω	3	1/4W carbon film	LED current limiting
Resistor 10kΩ	3	1/4W carbon film	Button pull-down
Breadboard	1	Full-size or half-size	Component mounting
Wires			Connections

9.0 C++ Code

9.1 Key Code Features

- 9-state FSM with enum-based state management
- Hardware interrupt for zero-latency earthquake detection
- Pulse-counting analysis over 5-second window
- Dynamic threshold calculation: map(sensitivityValue, 0, 1023, 15, 4) and map(sensitivityValue, 0, 1023, 25, 8)
- Test mode with tremorPulseCount = 30 (guarantees MAJOR classification)
- Non-blocking servo control using state-based progression (emergencyStep, resetStep)
- Standard LCD parallel interface on pins 12, 11, 13, A1, A2, A3
- Multi-modal output (LCD, LEDs, buzzer)
- Comprehensive serial debugging output

9.2 Source Code

```
#include <LiquidCrystal.h>
#include <Servo.h>

#define EARTHQUAKE_BUTTON  2
#define RESET_BUTTON       3
#define TEST_BUTTON        4
#define SENSITIVITY_POT    A0

#define YELLOW_LED         5
#define ORANGE_LED         6
#define RED_LED            7

#define BUZZER_PIN         8
#define GAS_SERVO_PIN      9
#define DOOR_SERVO_PIN     10

#define COUNTDOWN_DURATION  10
#define EMERGENCY_DELAY     2000
#define MINOR_DISPLAY_TIME  3000
#define DEBOUNCE_DELAY      50
#define LCD_UPDATE_INTERVAL 500
#define ANALYSIS_WINDOW     5000

#define GAS_VALVE_OPEN     0
#define GAS_VALVE_CLOSED   90
#define DOOR_LOCKED        0
#define DOOR_UNLOCKED      180

#define FREQ_MINOR         1000
#define FREQ_MODERATE      1500
#define FREQ_MAJOR         2000
#define FREQ_SAFE_MODE     500

enum SystemState {
  STANDBY,
  TREMOR_DETECTED,
  ANALYZING,
  MINOR_QUAKE,
  MODERATE_QUAKE,
  MAJOR_QUAKE,
  EMERGENCY_ACTIVE,
  SAFE_MODE,
```

```
  SYSTEM_RESETTING
};

SystemState currentState = STANDBY;
SystemState previousState = STANDBY;

volatile bool earthquakeTriggered = false;
volatile unsigned long lastInterruptTime = 0;

unsigned long tremorStartTime = 0;
unsigned long stateChangeTime = 0;
unsigned long lastCountdownUpdate = 0;
unsigned long lastBeepTime = 0;
unsigned long lastLCDUpdate = 0;

bool resetButtonPressed = false;
bool testButtonPressed = false;
bool lastResetState = false;
bool lastTestState = false;
unsigned long lastDebounceTime = 0;

unsigned long tremorDuration = 0;
int earthquakeSeverity = 0;
int countdownSeconds = COUNTDOWN_DURATION;
int tremorCount = 0;
int tremorPulseCount = 0;

int emergencyStep = 0;
int resetStep = 0;

bool testMode = false;
bool emergencyProtocolsActivated = false;
bool lcdNeedsUpdate = true;
bool minorBeepPlayed = false;

int sensitivityValue = 0;
int lastSensitivityDisplay = -1;

LiquidCrystal lcd(12, 11, 13, A1, A2, A3);
Servo gasValveServo;
Servo doorLockServo;

void changeState(SystemState newState);
String getStateName(SystemState state);
void readInputs();
void handleStandby();
void handleTremorDetected();
void handleAnalyzing();
void handleMinorQuake();
void handleModerateQuake();
void handleMajorQuake();
void handleEmergencyActive();
void handleSafeMode();
void handleSystemResetting();

void setup() {
  Serial.begin(9600);
  Serial.println("Earthquake Early Warning System v1.0 Initializing...");

  pinMode(EARTHQUAKE_BUTTON, INPUT);
  pinMode(RESET_BUTTON, INPUT);
  pinMode(TEST_BUTTON, INPUT);

  attachInterrupt(digitalPinToInterrupt(EARTHQUAKE_BUTTON), tremorISR, RISING);

  pinMode(YELLOW_LED, OUTPUT);
  pinMode(ORANGE_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(ORANGE_LED, LOW);
  digitalWrite(RED_LED, LOW);
  noTone(BUZZER_PIN);

  gasValveServo.attach(GAS_SERVO_PIN);
  doorLockServo.attach(DOOR_SERVO_PIN);
  gasValveServo.write(GAS_VALVE_OPEN);
  doorLockServo.write(DOOR_LOCKED);

  lcd.begin(16, 2);
```

```
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("EQ Warning Sys");
  lcd.setCursor(0, 1);
  lcd.print("Initializing...");
  delay(2000);

  lastBeepTime = millis();
  lastLCDUpdate = millis();

  changeState(STANDBY);
  Serial.println("System Ready!");
}

void loop() {
  readInputs();

  switch (currentState) {
    case STANDBY:
      handleStandby();
      break;
    case TREMOR_DETECTED:
      handleTremorDetected();
      break;
    case ANALYZING:
      handleAnalyzing();
      break;
    case MINOR_QUAKE:
      handleMinorQuake();
      break;
    case MODERATE_QUAKE:
      handleModerateQuake();
      break;
    case MAJOR_QUAKE:
      handleMajorQuake();
      break;
    case EMERGENCY_ACTIVE:
      handleEmergencyActive();
      break;
    case SAFE_MODE:
      handleSafeMode();
      break;
    case SYSTEM_RESETTING:
      handleSystemResetting();
      break;
    default:
      Serial.println("ERROR: Invalid state! Resetting to STANDBY");
      changeState(STANDBY);
      break;
  }

  if (resetButtonPressed && currentState != STANDBY && currentState != SYSTEM_RESETTING) {
    changeState(SYSTEM_RESETTING);
    resetButtonPressed = false;
  }
}

void tremorISR() {
  unsigned long interruptTime = millis();
  if (interruptTime - lastInterruptTime > 50) {
    earthquakeTriggered = true;
    lastInterruptTime = interruptTime;
  }
}

void readInputs() {
  unsigned long currentTime = millis();

  bool resetReading = digitalRead(RESET_BUTTON);
  if (resetReading != lastResetState) {
    lastDebounceTime = currentTime;
  }
  if ((currentTime - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (resetReading == HIGH && !resetButtonPressed) {
      resetButtonPressed = true;
    } else if (resetReading == LOW) {
      resetButtonPressed = false;
    }
  }
  lastResetState = resetReading;
```

```
  bool testReading = digitalRead(TEST_BUTTON);
  if (testReading != lastTestState) {
    lastDebounceTime = currentTime;
  }
  if ((currentTime - lastDebounceTime) > DEBOUNCE_DELAY) {
    if (testReading == HIGH && !testButtonPressed) {
      testButtonPressed = true;
    } else if (testReading == LOW) {
      testButtonPressed = false;
    }
  }
  lastTestState = testReading;

  sensitivityValue = analogRead(SENSITIVITY_POT);
}

void handleStandby() {
  int currentSensitivity = map(sensitivityValue, 0, 1023, 0, 100);
  if (lcdNeedsUpdate || currentSensitivity != lastSensitivityDisplay) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("EQ Monitor: SAFE");
    lcd.setCursor(0, 1);
    lcd.print("Sensitivity: ");
    lcd.print(currentSensitivity);
    lcd.print("%  ");
    lastSensitivityDisplay = currentSensitivity;
    lcdNeedsUpdate = false;
    lastLCDUpdate = millis();
  }

  if (earthquakeTriggered) {
    earthquakeTriggered = false;
    changeState(TREMOR_DETECTED);
    tremorStartTime = millis();
    tremorCount++;
    Serial.print("Earthquake detected! Count: ");
    Serial.println(tremorCount);
  }

  if (testButtonPressed) {
    Serial.println("Test mode activated");
    testMode = true;
    changeState(TREMOR_DETECTED);
    tremorStartTime = millis();
  }
}

void handleTremorDetected() {
  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Detecting...");
    lcd.setCursor(0, 1);
    lcd.print("Analyzing Data");
    lcdNeedsUpdate = false;
  }
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(ORANGE_LED, LOW);
  digitalWrite(RED_LED, LOW);
  changeState(ANALYZING);
}

void handleAnalyzing() {
  unsigned long currentTime = millis();
  tremorDuration = currentTime - tremorStartTime;

  if (currentTime - lastLCDUpdate >= LCD_UPDATE_INTERVAL || lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Analyzing...");
    lcd.setCursor(0, 1);
    lcd.print("Time left: ");
    lcd.print((ANALYSIS_WINDOW - tremorDuration) / 1000);
    lcd.print("s  ");
    lastLCDUpdate = currentTime;
    lcdNeedsUpdate = false;
  }

  if (earthquakeTriggered) {
    tremorPulseCount++;
```

```
    earthquakeTriggered = false;
    Serial.print("Vibration pulse counted: ");
    Serial.println(tremorPulseCount);
  }

  if (testMode) {
    tremorPulseCount = 30;
  }

  if (tremorDuration >= ANALYSIS_WINDOW) {
    Serial.print("Total Pulses Detected: ");
    Serial.println(tremorPulseCount);

    int moderateThreshold = map(sensitivityValue, 0, 1023, 15, 4);
    int majorThreshold = map(sensitivityValue, 0, 1023, 25, 8);

    Serial.print("Dynamic Moderate Threshold: ");
    Serial.println(moderateThreshold);
    Serial.print("Dynamic Major Threshold: ");
    Serial.println(majorThreshold);

    if (tremorPulseCount < moderateThreshold) {
      earthquakeSeverity = 1;
      changeState(MINOR_QUAKE);
    } else if (tremorPulseCount < majorThreshold) {
      earthquakeSeverity = 2;
      changeState(MODERATE_QUAKE);
      countdownSeconds = COUNTDOWN_DURATION;
      lastCountdownUpdate = millis();
    } else {
      earthquakeSeverity = 3;
      changeState(MAJOR_QUAKE);
      countdownSeconds = COUNTDOWN_DURATION;
      lastCountdownUpdate = millis();
    }

    Serial.print("Earthquake classified as: ");
    Serial.println(earthquakeSeverity == 1 ? "MINOR" :
                   earthquakeSeverity == 2 ? "MODERATE" : "MAJOR");

    testMode = false;
    tremorPulseCount = 0;
  }
}

void handleMinorQuake() {
  digitalWrite(YELLOW_LED, HIGH);
  digitalWrite(ORANGE_LED, LOW);
  digitalWrite(RED_LED, LOW);

  if (!minorBeepPlayed) {
    tone(BUZZER_PIN, FREQ_MINOR, 200);
    minorBeepPlayed = true;
  }

  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Minor Tremor    ");
    lcd.setCursor(0, 1);
    lcd.print("Duration: ");
    lcd.print(tremorDuration / 1000);
    lcd.print("s  ");
    lcdNeedsUpdate = false;
  }

  if (millis() - stateChangeTime > MINOR_DISPLAY_TIME) {
    digitalWrite(YELLOW_LED, LOW);
    noTone(BUZZER_PIN);
    minorBeepPlayed = false;
    changeState(STANDBY);
  }
}

void handleModerateQuake() {
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(ORANGE_LED, HIGH);
  digitalWrite(RED_LED, LOW);

  if (millis() - lastCountdownUpdate >= 1000) {
    countdownSeconds--;
```

```
    lastCountdownUpdate = millis();
    lcdNeedsUpdate = true;
  }

  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("MODERATE QUAKE! ");
    lcd.setCursor(0, 1);
    lcd.print("Peak in: ");
    lcd.print(countdownSeconds);
    lcd.print("s   ");
    lcdNeedsUpdate = false;
  }

  unsigned long currentTime = millis();
  unsigned long beepCycle = (currentTime - lastBeepTime) % 2000;
  if (beepCycle < 100 || (beepCycle >= 200 && beepCycle < 300) || (beepCycle >= 400 && beepCycle
< 500)) {
    tone(BUZZER_PIN, FREQ_MODERATE);
  } else {
    noTone(BUZZER_PIN);
  }

  if (countdownSeconds <= 0) {
    noTone(BUZZER_PIN);
    digitalWrite(ORANGE_LED, LOW);
    changeState(STANDBY);
  }
}

void handleMajorQuake() {
  digitalWrite(YELLOW_LED, LOW);
  digitalWrite(ORANGE_LED, LOW);
  digitalWrite(RED_LED, HIGH);

  if (millis() - lastCountdownUpdate >= 1000) {
    countdownSeconds--;
    lastCountdownUpdate = millis();
    lcdNeedsUpdate = true;
  }

  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("MAJOR QUAKE!!!  ");
    lcd.setCursor(0, 1);
    lcd.print("Peak in: ");
    lcd.print(countdownSeconds);
    lcd.print("s   ");
    lcdNeedsUpdate = false;
  }

  tone(BUZZER_PIN, FREQ_MAJOR);

  if (countdownSeconds <= 0) {
    changeState(EMERGENCY_ACTIVE);
  }
}

void handleEmergencyActive() {
  digitalWrite(RED_LED, HIGH);
  unsigned long timeInState = millis() - stateChangeTime;

  if (emergencyStep == 0) {
    Serial.println("Closing gas valve...");
    gasValveServo.write(GAS_VALVE_CLOSED);
    emergencyStep = 1;
  }
  else if (emergencyStep == 1 && timeInState >= 300) {
    Serial.println("Unlocking emergency doors...");
    doorLockServo.write(DOOR_UNLOCKED);
    emergencyStep = 2;
  }

  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Gas:OFF Door:OPEN");
    lcd.setCursor(0, 1);
    lcd.print("EMERGENCY MODE ");
```

```
    lcdNeedsUpdate = false;
  }

  unsigned long beepCycle = (millis() - lastBeepTime) % 1000;
  if (beepCycle < 500) {
    tone(BUZZER_PIN, FREQ_SAFE_MODE);
  } else {
    noTone(BUZZER_PIN);
  }

  if (timeInState > EMERGENCY_DELAY) {
    changeState(SAFE_MODE);
    emergencyStep = 0;
  }
}

void handleSystemResetting() {
  unsigned long timeInState = millis() - stateChangeTime;

  if (resetStep == 0) {
    Serial.println("PERFORMING SYSTEM RESET...");
    digitalWrite(YELLOW_LED, LOW);
    digitalWrite(ORANGE_LED, LOW);
    digitalWrite(RED_LED, LOW);
    noTone(BUZZER_PIN);

    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("System Reset    ");
    lcd.setCursor(0, 1);
    lcd.print("Returning...    ");

    Serial.println("Reopening gas valve...");
    gasValveServo.write(GAS_VALVE_OPEN);
    resetStep = 1;
  }
  else if (resetStep == 1 && timeInState >= 300) {
    Serial.println("Locking doors...");
    doorLockServo.write(DOOR_LOCKED);
    resetStep = 2;
  }
  else if (resetStep == 2 && timeInState >= 1500) {
    earthquakeSeverity = 0;
    tremorDuration = 0;
    minorBeepPlayed = false;
    resetStep = 0;

    Serial.println("System reset complete!");
    changeState(STANDBY);
  }
}

void handleSafeMode() {
  digitalWrite(RED_LED, HIGH);
  if (lcdNeedsUpdate) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Press RESET to  ");
    lcd.setCursor(0, 1);
    lcd.print("Resume Monitor");
    lcdNeedsUpdate = false;
  }

  unsigned long currentTime = millis();
  unsigned long beepCycle = (currentTime - lastBeepTime) % 2000;
  if (beepCycle < 300) {
    tone(BUZZER_PIN, FREQ_SAFE_MODE);
  } else {
    noTone(BUZZER_PIN);
  }
}

void changeState(SystemState newState) {
  previousState = currentState;
  currentState = newState;
  stateChangeTime = millis();
  lcdNeedsUpdate = true;
  lastBeepTime = millis();

  Serial.print("State Change: ");
  Serial.print(getStateName(previousState));
```

```
  Serial.print(" -> ");
  Serial.println(getStateName(currentState));
}

String getStateName(SystemState state) {
  switch (state) {
    case STANDBY: return "STANDBY";
    case TREMOR_DETECTED: return "TREMOR_DETECTED";
    case ANALYZING: return "ANALYZING";
    case MINOR_QUAKE: return "MINOR_QUAKE";
    case MODERATE_QUAKE: return "MODERATE_QUAKE";
    case MAJOR_QUAKE: return "MAJOR_QUAKE";
    case EMERGENCY_ACTIVE: return "EMERGENCY_ACTIVE";
    case SAFE_MODE: return "SAFE_MODE";
    case SYSTEM_RESETTING: return "SYSTEM_RESETTING";
    default: return "UNKNOWN";
  }
}
```

10.0 Conclusion

The QuakeGuard Intelligent Earthquake Early Warning System successfully demonstrates advanced embedded systems concepts including hardware interruptions, finite state machines, pulse-counting analysis, and non-blocking servo control. The system addresses real-world earthquake response challenges through automated gas shutoff and emergency door unlocking, potentially reducing post-disaster casualties from fires and building entrapment.

The project achieved all primary objectives through its 9-state Finite State Machine, which provides robust state management from standby through emergency response and reset. Zero-latency detection via hardware interrupts eliminates missed vibration events, ensuring no seismic activity goes unnoticed. Intelligent classification through pulse-counting over a 5-second window outperforms simple duration-based detection methods, accurately distinguishing minor tremors from dangerous seismic events. Non-blocking servo control executes emergency protocols including gas valve shutoff and door unlocking without freezing system responsiveness, allowing the system to remain aware of aftershocks during critical operations.

Future enhancements may include wireless multi-building coordination for community-wide alerting, persistent timestamp logging for event analysis and record-keeping, and real seismographic sensor integration to replace simulated button inputs. These additions would further extend the system's capabilities toward real-world deployment.

The QuakeGuard project demonstrates that effective earthquake early warning and automated response is achievable using affordable, widely available components, making building-level seismic protection accessible for residential and commercial applications.